EC-210 Complex Engineering Problem: High-Performance Virtual Memory & TLB Simulator

Report due: May 03, 2026 (Sunday),

Project Presentations & Viva: May 05, 2026 (Tuesday or before)

Students can work in groups of up to 3 students.

1. Project Overview

You are tasked with developing a software-based **Memory Management Unit (MMU)**. This simulator must model how an Operating System translates 32-bit virtual addresses into physical addresses using a **Page Table** and a **Translation Lookaside Buffer (TLB)**. A core requirement is the **Performance Modeling Engine**, where you will simulate hardware latencies to calculate the **Effective Access Time (EAT)** of the system.

2. Washington Accord Knowledge Profile (WK)

This project assesses your mastery of the following Knowledge Profile elements:

- **WK3: Engineering Fundamentals:** Application of bitwise operations and mathematical partitioning to map address spaces.
- **WK4: Specialist Knowledge:** Analysis of the "Dirty Bit" propagation and how software state impacts hardware I/O (Disk Writes).
- **WK6: Modern Tool Usage:** Use of version control (Git) and performance profiling to analyze system efficiency.

3. Washington Accord Professional Attributes (WP)

This project is classified as a **Complex Engineering Problem** based on:

- **WP1: Analysis of Conflicting Requirements:** You must balance **TLB Size** (Hardware Cost) vs. **EAT** (Performance) and **Page Size** (Table Overhead) vs. **Internal Fragmentation**.
- **WP2: Depth of Analysis Required:** You must provide a "Proof of Correctness" by manually tracing 10 instructions and proving that your simulated Latency Engine matches theoretical mathematical models.

4. Technical Specifications & Scalability

The simulator **must not** contain hard-coded memory values. All parameters must be loaded from an external **Configuration File** (JSON, YAML, or TXT).

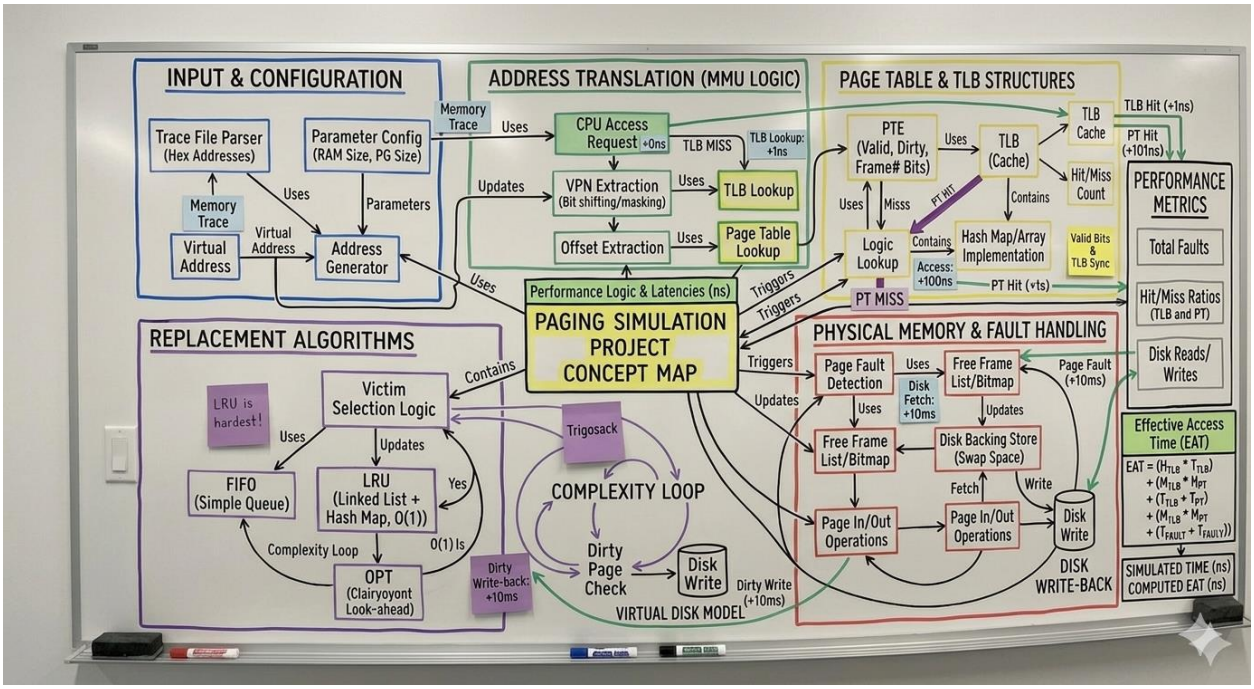
- **Scalability:** The system must be designed to handle scaling. For example, if the Page Size is decreased from 4 KB to 512 bytes, the Page Table must scale dynamically without memory corruption or $O(n^2)$ performance degradation.
- **Configurable Parameters:**
    - **Physical RAM Size:** (e.g., 256 KB to 2 GB).
    - **Page Size:** (e.g., 1 KB to 64 KB).
    - **TLB Size:** (e.g., 4 to 512 entries).
    - **Latencies:** Custom values (in ns) for TLB, RAM, and Disk access.

Implementation Constraints

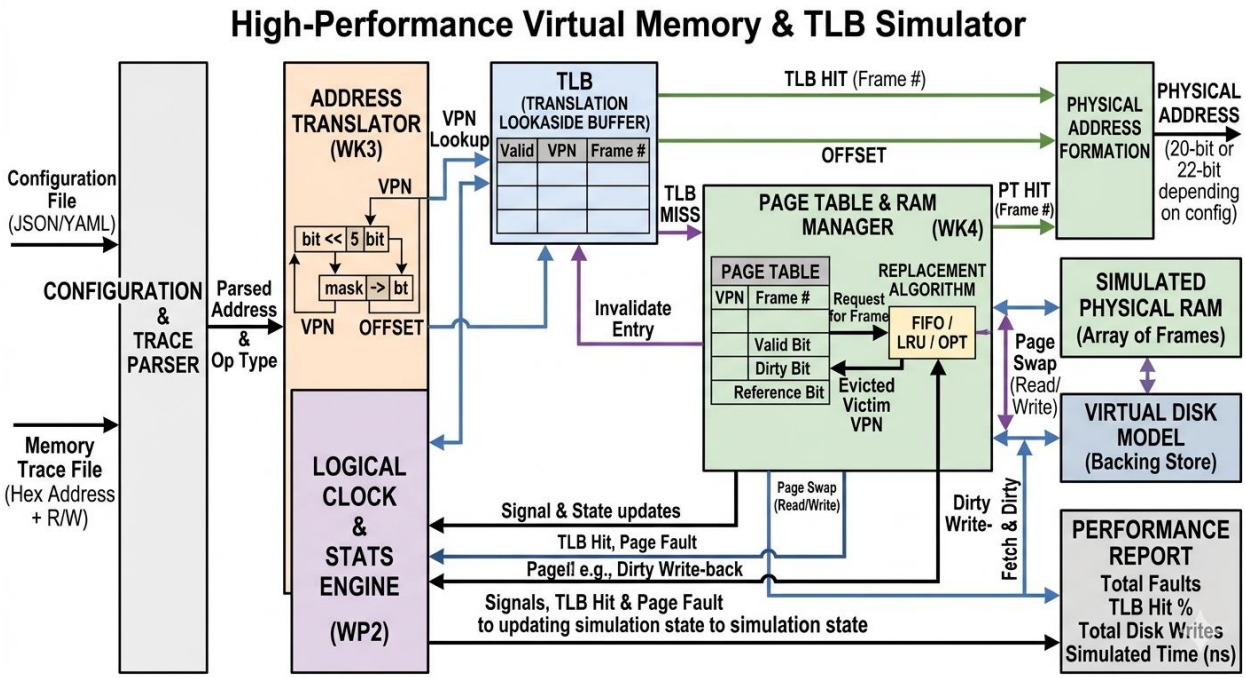
- **Language:** C, C++, or Python
- **The Trace File:** Must support at least 100,000 memory references.
- **Reporting:** A 2-page technical report comparing Algorithm performance vs. Frame Count (identifying **Belady's Anomaly**).

4. System Design & Architectural Logic

Study these two perspectives before beginning your implementation. They differentiate between **what** the system is (the blueprint) and **how** it behaves (the logic).



Proposed Diagram I: The System Blueprint/ Block Diagram



Proposed Diagram II: The Logic Flow & Latency Engine

5. Deliverables (The 10-Step Roadmap)

Note: Your implementation style can vary. But all the components must be there.

Level	Deliverable	Description
Easy	1. Config Parser	Read System Parameters: Page Size (e.g., 4KB), Physical RAM size.
	2. Address Splitter	Extract **VPN** and **Offset** using bit-masking ($0xFFF$) and shifting ($>> 12$).
	3. Trace Validator	Handle input errors and identify if an address is "Out of Bounds."
Mid	4. Page Table	Implement a structure tracking Valid Bits and Frame Numbers.
	5. Free Frame List	Manage a list/bitmap of available physical memory slots.
	6. FIFO Algorithm	Implement First-In-First-Out eviction logic.
	7. Stats Engine	Real-time tracking of Page Faults, Hits, and Fault Rate (%).

Hard	8. Dirty Bit Logic	Track "Write" ops; trigger a "Disk Write" only if the evicted page was modified.
	9. LRU (O(1))	Implement Least Recently Used using a Doubly Linked List + Hash Map.
	10. OPT Algorithm	A "Clairvoyant" algorithm that pre-scans the trace to find the optimal victim.

6. Detailed Implementation Guidelines

i. Input & Configuration Module

- **Description:** The gateway that initializes the simulation environment by reading parameters (RAM Size, Page Size) and feeding virtual addresses from the Trace File.
- **Guidelines:** * Initialize a SystemConfig object or struct to store physical frame counts and bit-shift values.
    - The **Trace File Parser** must read line-by-line (not all at once) to handle the 100,000+ references efficiently.
- **Suggestion:** Implement an **Address Generator** helper function that cleans the hex strings (e.g., stripping 0x) and converts them to unsigned integers before passing them to the translation logic.

ii. Address Translation (MMU Logic)

- **Description:** Simulates the Memory Management Unit (MMU) by splitting a CPU access request into its component parts: VPN and Offset.
- **Guidelines:** * Calculate the shift amount based on the page size (21^2 = 4KB}, so shift is 12).
    - **VPN Extraction:** Use address >> shift.
    - **Offset Extraction:** Use address & (pageSize - 1).
- **Suggestion:** Even if you aren't required to use the Offset for the simulation logic, print it during debugging to ensure your bit-masking is working correctly.

iii. Page Table & TLB Structures

- **Description:** The data structures that map virtual addresses to physical frame numbers.

- **Guidelines:** * The **PTE (Page Table Entry)** must contain at least: Valid bit, Dirty bit, and Frame Number.
    - If implementing a **TLB (Cache)**, use a small fixed-size array or list to check for hits *before* searching the main Page Table.
- **Suggestion:** Use an unordered_map (C++) or a Dictionary (Python) for the Page Table to handle sparse memory addresses without wasting RAM on unallocated entries.

iv. Replacement Algorithms

- **Description:** The "Victim Selection Logic" that decides which page to evict when physical memory is full.
- **Guidelines:**
    - **FIFO:** Use a simple queue of VPNs.
    - **LRU:** To achieve O(1) as noted in your diagram, use a Doubly Linked List + Hash Map.
    - **OPT:** Requires a pre-scan of the trace file to build a "future-use" dictionary.
- **Suggestion:** Start with FIFO to test your basic fault-handling logic, then move to LRU once you've confirmed that pages are being evicted correctly.

v. Physical Memory & Fault Handling

- **Description:** Manages the available physical frame slots and triggers "Page In" operations from the virtual disk.
- **Guidelines:**
    - Maintain a **Free Frame List** (or Bitmap). If a frame is available, assign it directly.
    - If not, trigger the Replacement Algorithm, check the **Dirty Bit**, and perform a "Disk Write" if necessary before bringing in the new page.
- **Suggestion:** Track the "Free Frame List" as a simple stack of integers representing frame indices (e.g., 0 to N-1).

vi. Virtual Disk Model (Write-Back)

- **Description:** Simulates the secondary storage.
- **Guidelines:** * This is mostly a logical counter. You don't actually write to disk; you just track the latency and the number of "Dirty Page" evictions.

- A "Dirty Write" adds significantly to your simulated time (e.g., +10\text{ms} per the diagram).

- **Suggestion:** Ensure your **Dirty Page Check** happens *immediately* before the page is evicted to accurately count disk traffic.

7. Performance Metrics & EAT

- **Description:** The final report generator that computes the efficiency of your simulation.
- **Guidelines:**
  - Keep counters for Total Faults, Total Hits, and Disk Reads/Writes.
  - **Effective Access Time (EAT):** Implement the formula like the one shown in your diagram to provide a technical summary of the system performance.
- **Suggestion:** Print the results in a clean table format. For 100,000 references, even a 1% difference in the hit ratio can significantly impact the final EAT.

6. Module-Wise Evaluation Rubric

Module	**Criteria**	**Marks**
I. Input & Config	Correct bit-masking/shifting for VPN extraction; no string-parsing of addresses.	10
II. TLB Logic	Accurate Hit/Miss logic; **Mandatory Invalidation** of TLB entries upon RAM eviction.	10
III. RAM & Page Table	Correct state management (Valid/Dirty bits); accurate Free Frame list tracking.	15
IV. Latency Engine	Logical clock correctly accounts for sequential costs (e.g., TLB miss still costs 1ns).	15
V. Algorithms	Successful implementation of FIFO, O(1) LRU, and OPT (look-ahead) logic.	20
VI. System Integration	End-to-end data flow; inter-module communication, memory leak checks; handling 100k+ references.	20
VII. Analysis Report	Professional comparison of EAT; clear identification of Belady's Anomaly. Simulation vs manual tracing for 10 instructions.	10
TOTAL		**100**